Basics of Python

01 January 2024 15:30

Keywords

```
import keyword
print(keyword.kwlist)
print(keyword.softkwlist)
# ['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
# 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for',
#  'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or',
# 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

✓ 0.0s

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'cont
['_', 'case', 'match']
```

statements

```
# multi-line statements
print("hello \
there")

# multiple statements in a single line
print("hello"); print("there")
```

✓ 0.0s

```
hello there
hello
there
```

```
x = 3
print(id(x))
y = 3
print(id(y))
```

✓ 0.0s

```
10781736
10781736
```

Here, the same memory location is used for x and y. Since a memory location with 3 already exists, it is reused and y points to the same.
If we change y, it'll point somewhere else.

```
# data types

# complex
a = 3+4j
b = 2+3j
print(type(a))
print(a+b)

# float a = 5.1, int a = 5
print(isinstance(True, bool))
# strings are unicode in Python
```

✓ 0.0s

```
<class 'complex'>
(5+7j)
True
```

sets: unordered, unique { 1, 3, 4 }
lists: ordered, mutable [1, 3, 4]
tuples: ordered, immutable (1, 3, 4)
dict: unordered, key-value pairs { 'melons': 12, 'oranges': 23 }

Type conversion:
a = 5 # int
a = float(a)

str.format()

```
a = 'first'; b = 'second'
print('{}'.format(a))
print('{1} {0} {c}'.format(a, b, c='third'))
```

✓ 0.0s

```
first
second first third
```

```
# list indexing
# list_name[start:end:step]
[ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11 ][3:8:2]
```

✓ 0.0s

```
[4, 6, 8]
```

```
# set operations
s1 = {1, 2, 3, 4}
s2 = {11, 22, 33, 1}
s1.update([4, 5, 6, 7, 4, 5])
print(s1)
s1.remove(4)
s1.discard(5)
print("\ns1: ", s1, "\ns2: ", s2, "\n")
print(s1 | s2)  # union, same as s1.union(s2)
print(s1 & s2)  # intersection, s1.intersection(s2)
print(s1 - s2)  # set difference, s1.difference(s2)
print(s1 ^ s2)  # s1.symmetric_difference(s2), like XOR
print(s1.issuperset({2, 3}))
print(s1.issubset({2, 3}))
```

✓ 0.0s

```
{1, 2, 3, 4, 5, 6, 7}

s1:  {1, 2, 3, 6, 7}
s2:  {33, 11, 22, 1}

{1, 2, 3, 33, 6, 7, 11, 22}
{1}
{2, 3, 6, 7}
{33, 2, 3, 6, 7, 11, 22}
True
False
```

```
# frozen sets are immutable
# mutable sets are not hashable, so can't be used as keys to a dictionary
# frozen sets are hashable, so can be used as keys to a dictionary
fs1 = frozenset({1, 2, 3, 4})
print(fs1)
```

✓ 0.0s

```
frozenset({1, 2, 3, 4})
```

```
firstdict = { 'one': 1, 'two': 2, 'three': 3 }
```

```
    print(firstdict)
    firstdict['four'] = 4
    print(firstdict)
    seconddict = { k: v**2 for k, v in firstdict.items() }
    print(seconddict)
```

✓ 0.0s

```
{'one': 1, 'two': 2, 'three': 3}
{'one': 1, 'two': 2, 'three': 3, 'four': 4}
{'one': 1, 'two': 4, 'three': 9, 'four': 16}
```

Strings are immutable.
Operations are +(concatenation), *(repetition)
Strings are iterable.
Membership operator: in
String methods: lower(), upper(), join(), split() splits on given character, find() gives index,

Functions:

```
# functions

def foo(nm='sanved'):
    '''says hello there '''
    print("hello there")

foo()
print(foo.__doc__)
print(foo.__defaults__)
```

✓ 0.0s

```
hello there
says hello there
('sanved',)
```

```
l1 = [0, 1, 4, 9, 16, 25]

for index, num in enumerate(l1):
    print(f"{index}: {num}")

print(list(filter(None, l1)))
print(list(filter(lambda x : x > 10, l1)))

print(isinstance([], list)) # isinstance(obj, class)

print(list(map(lambda x: x*x, l1)))

from functools import reduce
print(reduce(lambda x, y: x+y, l1))
# reduce(function, iterable[, initial]) -> value
```

✓ 0.0s

```
0: 0
1: 1
2: 4
3: 9
4: 16
5: 25
[1, 4, 9, 16, 25]
[16, 25]
True
[0, 1, 16, 81, 256, 625]
55
```

```
def foo(**kwargs):
    print(f"hello {kwargs['name']}")

foo(name="sanved")

def foo2(*names):
    for name in names:
        print(f"hello {name}")

foo2('abc', 'pqr', 'xyz')
```

```
✓ 0.0s
hello sanved
hello abc
hello pqr
hello xyz
```

Files:

sample.txt:
hi

```
fp = open('sample.txt', 'r')
print(f"original file:\n{fp.read()}")
fp.close()

fp = open('sample.txt', 'a')
fp.write('hello there')
fp.close()

fp = open('sample.txt', 'r')
print(f"modified file:\n{fp.read()}")
fp.close()

fp = open('sample.txt', 'r')
print(f"\nsame file with only first 4 characters:\n{fp.read(4)}")
fp.close()
```

```
[100] ✓ 0.0s
original file:
hi

modified file:
hi
hello there

same file with only first 4 characters:
hi
h
```

after modification, sample.txt:
hi
hello there

```
f = open('sample.txt', 'r')

f.seek(1)

for line in f:
    print(line)
```

```
[101] ✓ 0.0s
i

hello there
```

```
f.seek(0)
print(f.readline())
print(f.readline())
print("----")
f.seek(0)
print(f.readlines())
```

```
✓ 0.0s
hi

hello there
----
['hi\n', 'hello there']
```

```
import os
os.rename('sample.txt', 'test.txt')
os.remove('test.txt')
```

```
✓ 0.0s
```

Directory Management:

```
print(os.getcwd())
```

✓ 0.0s

```
/home/sanved/ml/M1
```

```
os.chdir('/home/sanved/ml/M1/d1')
print(os.getcwd())
os.chdir('/home/sanved/ml/M1')
print(os.listdir(os.getcwd()))
```

✓ 0.0s

```
/home/sanved/ml/M1/d1
['fundamentals_python.ipynb', 'one.txt', 'd1']
```

```
os.mkdir('d2')
print(os.listdir(os.getcwd()))
os.rmdir('d2')  # can't remove non-empty dirs
print(os.listdir(os.getcwd()))

os.mkdir('d2')
# to remove non-empty directory
import shutil
shutil.rmtree('d2')
```

✓ 0.0s

```
['fundamentals_python.ipynb', 'one.txt', 'd1', 'd2']
['fundamentals_python.ipynb', 'one.txt', 'd1']
```

Debugging

```
# debugging
import pdb

# q:quit, c:continue, h:help, p:print, list
# examples:
# p i, n
# p locals()

def foo(n):
    for i in range(n):
        pdb.set_trace()
        print(i+1)

foo(5)
```

Gives you an interactive window at set_trace
You can check values of variables every time at every execution
Functions:

- h: help
- w: where
- locals(): gives local variables
- etc.

Ex. p i, n would print the pair (i, n) (their values at the time of execution of that statement)

Numpy

Importing

```
import numpy as np
```

```
import numpy as np
```

```
a = np.array([[1, 2, 7], [3, 4, 8], [5, 6, 9]])
print(a, "\n")

print(np.arange(1, 10, 2))  # (start, stop, step)
print(np.arange(10))
print(np.linspace(0, 10, 6))    # (start, stop, num)
print("\n", np.eye(3, 3))
# np.ones, np.zeros

print("\n", np.diag(a))
```

✓ 0.0s

```
[[1 2 7]
 [3 4 8]
 [5 6 9]]

[1 3 5 7 9]
[0 1 2 3 4 5 6 7 8 9]
[ 0.  2.  4.  6.  8. 10.]

 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

 [1 4 9]
```

```
np.random.rand(2,3)
```

✓ 0.0s

```
array([[0.0385427 , 0.26912646, 0.65065033],
       [0.52179687, 0.31410039, 0.82675386]])
```

```
a = np.array([1, 2, 3, 4])
b = a[::2]
print(b)
print(np.shares_memory(a, b))
```

✓ 0.0s

```
[1 3]
True
```

All arithmetic operations with scalars are performed element-wise
E.g. If a = [1, 2, 3]
Then a+3=[4, 5, 6]
If b = [4, 5, 6]
Then a*b = [4, 10, 18] (element-wise)
Dot product using np.dot(a,b)

Masking indices
Like arr[arr>2]

Use np.array_equal(a, b) to compare 2 arrays

Logical operators like logical_and(a, b), logical_or(a, b)

Tile function tiles rows or columns (duplicates elements)

```
a = [1, 2, 3]
print(np.tile(a, (4, 1)))
```

✓ 0.0s

```
[[1 2 3]
 [1 2 3]
```

```
[1 2 3]
[1 2 3]]
```

```
print(np.tile(a, 2))
```
✓ 0.0s
```
[1 2 3 1 2 3]
```

```
a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
b = np.array([[2, 3, 4]])

print(a+b)
```
✓ 0.0s
```
[[ 3  5  7]
 [ 6  8 10]
 [ 9 11 13]]
```

here, tiling takes place implicitly
a is
1 2 3
4 5 6
7 8 9
b becomes
2 3 4
2 3 4
2 3 4

This is called broadcasting

If a is
1 2 3
4 5 6
7 8 9
And b is
1
2
3
Then b becomes
1 1 1
2 2 2
3 3 3

Flattening:

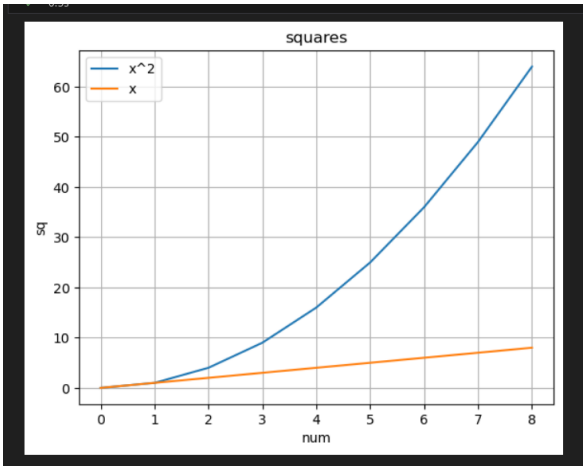
```
a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(a.ravel())
```
✓ 0.0s
```
[1 2 3 4 5 6 7 8 9]
```

Also the reshape function can be used

Matplotlib

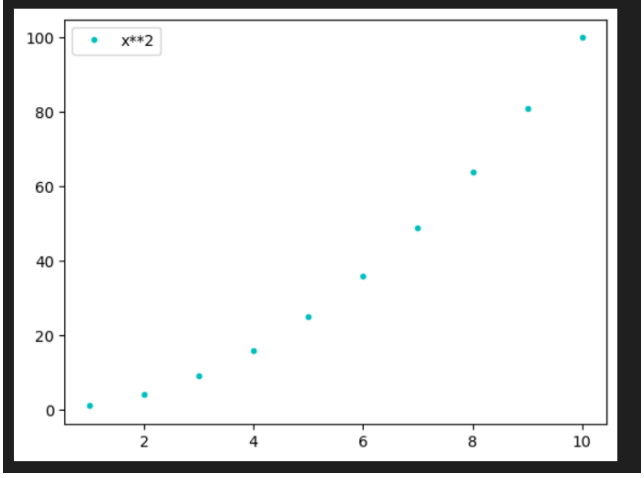
```
import matplotlib.pyplot as plt
```

```
plt.plot([0, 1, 4, 9, 16, 25, 36, 49, 64])
plt.plot([0, 1, 2, 3, 4, 5, 6, 7, 8])
plt.xlabel("num")
plt.ylabel("sq")
plt.title("squares")
plt.legend(["x^2", "x"])
plt.grid()
plt.show()
```
✓ 0.5s




```
x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
plt.plot(x, x**2, 'c.', label='x**2')
plt.legend()
plt.show()
# . = dots
# ^ = triangles
# -- = dashed line
# o = circles
# s = squares
# can be used with colors, like b-- gives blue dashed lines
# here c. gives cyan dots
```

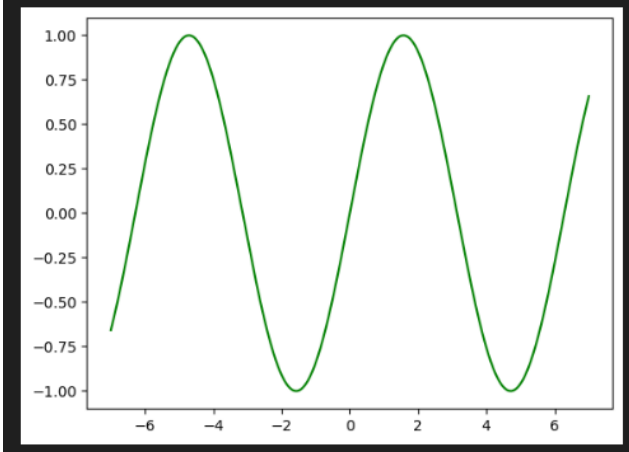
✓ 0.3s

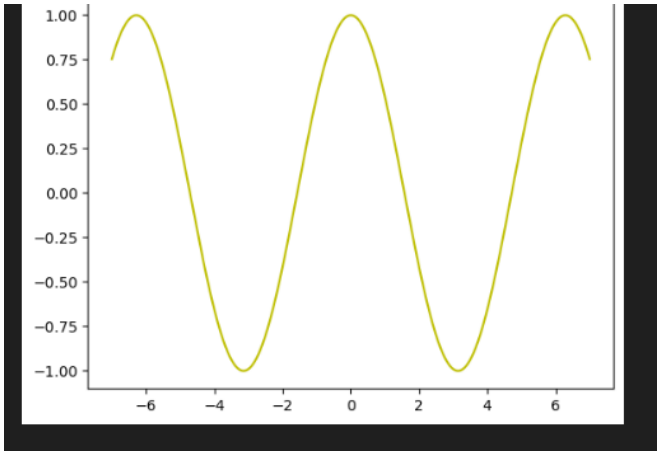



```
x = np.linspace(-7, 7, 10000)
plt.figure(1)
plt.plot(x, np.sin(x), 'g')
plt.figure(2)
plt.plot(x, np.cos(x), 'y')
```

✓ 0.2s

```
[<matplotlib.lines.Line2D at 0x7fd5faad62d0>]
```






```
plt.subplot(221)
plt.plot(x, x)
plt.plot(x, x**2)
plt.subplot(222)
plt.plot(x, x)
plt.plot(x, x**3)
plt.subplot(224)
plt.plot(x, np.sin(x))
plt.plot(x, np.cos(x))
```

✓ 0.4s

```
[<matplotlib.lines.Line2D at 0x7fd5fa511950>]
```

